

COMP5311 Lecture 1 / 2 / 3 Cheat Sheet (Application Layer 完整版)

整合: Lecture 1 (Introduction) + Lecture 2 (App Layer I) + Lecture 3 (App Layer II) 来源: PPT 原文 + Markdown 复习 + 对话补充。重要术语保留 PPT 原始表述。

1. Internet Fundamentals

1.1 What is the Internet (PPT 原话)

Internet = “**network of networks**”: interconnected ISPs.

组件	说明
hosts = end systems	运行 network apps 的设备 (PC、手机、server、IoT)
communication links	fiber, copper, radio, satellite; transmission rate = bandwidth
Packet switches	转发 packets, 包括 routers 和 switches
ISPs	Internet Service Providers, 互连成 Internet

Internet 提供: ① infrastructure for apps (Web, VoIP, email, ...) ② programming interface (socket API) ③ service options (类比 postal service)。

Internet Structure (PPT 原话): – End systems connect to Internet via **access ISPs** (residential, company, university ISPs) – Access ISPs in turn must be **interconnected**, so any two hosts can send packets to each other – Resulting network of networks is very complex; evolution was driven by **economics and national policies** – 关键互连组件: **IXP (Internet eXchange Point)**、**peering link** (ISP 之间直连的链路)

Communication Network 定义 (PPT 原话): > Communication networks are arrangements of **hardware and software** that allow users to exchange information. > A communication network is a **mesh connection of links and switches**, which permits the exchange of information.

1.2 Network Switch 数学

直连	用 switch
n 台设备两两连: $n(n-1)/2 = O(n^2)$ 条线	每台连到 switch: $O(n)$ 条线

1.3 Internet History 关键节点

- 1961 Kleinrock: queueing theory 验证 packet switching
- 1972 ARPAnet 公开演示 (NCP、首个 e-mail)
- 1974 Cerf & Kahn → TCP/IP (图灵奖 2005)
- Cerf & Kahn 设计原则 (PPT 原文): minimalism、autonomy、**best effort service model**、**stateless routers**、**decentralized control**
- 1976 Ethernet @ Xerox PARC
- 1982 SMTP; 1983 DNS、TCP/IP 部署; 1988 TCP congestion control
- 1990s Web (HTML/HTTP, Berners-Lee) → commercialization
- 2005-now: 5G、社交网络、CDN、cloud/edge

2. Protocols

PPT 定义: protocols define **format**, **order** of msgs sent and received among network entities/nodes, and **actions taken** on msg transmission, receipt.

三个关键词: **format (格式)** + **order (顺序)** + **actions (动作)**。

人类协议类比: Hi → Hi → Got the time? → 2:00, 对应 TCP 的 SYN → SYNACK → ACK → GET → file。

3. Layered Architecture

3.1 为什么分层 (PPT 原话)

Network functions are organized in a **layered architecture**. The services of one layer are implemented on top of the services provided by the layer **immediately below**.

优点: ① 各层 **independent** 设计 ② **compatibility** 保证不同网络互联 ③ 简化网络设计。

3.2 三种模型对照

7-layer OSI	5-layer (本课程主用)	4-layer Internet
Application	Application	Application
Presentation ❌	(合并到 Application)	(合并到 Application)
Session ❌	(合并到 Application)	(合并到 Application)
Transport	Transport	TCP or UDP (Transport)
Network	Network (IP)	IP / Internet layer
Data Link	Link	Network layer
Physical	Physical	(含在 Network)

OSI 中 Presentation/Session 在 Internet 实际上未实现 (Not implemented), 功能合并到 Application。

OSI 7 层的 3 个 subgroups (PPT 原话, 考点):

Subgroup	包含层	作用
Network support layers	Layers 1, 2, 3	处理“把数据从一个设备搬到另一个”的物理细节: specifications, physical connections, physical addressing, transport timing & reliability
User support layers	Layers 5, 6, 7	让不同软件系统 互操作 (interoperability among unrelated software systems)
链接层	Layer 4 (Transport)	链接 network support 和 user support 两个 subgroups, 确保下层传出的数据格式上层能用

OSI 中 Session / Presentation 各做什么 (5 层模型里被合并到 App, 但 OSI 题可能问): – **Session layer**: uses transport-layer services to set up and supervise connections between end systems – **Presentation layer**: data compression, security, format conversions (让用不同表示法的节点能高效安全通信)

3.3 五层全景表 (全课程跨三章总览, 必背)

下表把课程涉及的所有知识点按层归位——是 Cheat Sheet 的“全局地图”。

Layer	单位	寻址	核心问题 / 服务	关键概念 / 机制	协议 / 技术 (本课程)
Application	message	hostname / URL	应用进程之间通信, 定义消息格式	client-server vs P2P; socket API; persistent vs non-persistent HTTP; Web cache; DASH bitrate adaptation	HTTP/1.1, Cookies, Web Cache, DNS (iter./recur., RR: A/NS/CNAME/MX, DNSSEC), SMTP, IMAP, POP3, FTP, DHCP, P2P/BitTorrent, DASH+CDN
Transport	segment	port	进程到进程; 可靠/不可靠; 流量+拥塞控制	mux/demux; 3-way handshake; byte stream + cumulative ACK; GBN/SR; RTT 估算 (EWMA); timer + fast retransmit; rwnd 流量控制; cwnd + AIMD + slow start + ssthresh; Tahoe vs Reno; BBR; fairness	TCP, UDP
Network	datagram	IP (32/128-bit)	主机到主机; 跨网络路由; data/control plane	IP datagram; CIDR a.b.c.d/x; subnet; longest prefix matching + TCAM; router internals (input/fabric/output); HOL blocking + VOG; buffer mgmt + drop policy; scheduling (FCFS/Priority/RR/WFQ); NAT; IPv6 + tunneling; route aggregation; Dijkstra (LS) / DV; generalized forwarding; OpenFlow / SDN	IP (v4/v6), ICMP, OSPF (LS/Dijkstra), RIP (DV), BGP, OpenFlow
Link	frame	MAC	相邻节点之间传输; MAC↔IP 解析	frame format; CRC/checksum; broadcast; ARP query/reply	Ethernet (802.3), WiFi (802.11), ARP
Physical	bit	—	比特在介质中传输	bandwidth; signal; propagation delay	fiber, copper, radio

注意: **DHCP** 是 application-layer 协议 (用 UDP), 但**功能上**配置 IP / 网关 / DNS 等网络层信息——常考”DHCP 在哪一层”。

关键词 → 层 速判:

关键词	层
域名 / Web / 邮件 / 文件传输	Application
端口 / 可靠传输 / 流量控制 / 拥塞控制 / 三次握手	Transport
路由 / IP 地址 / 跨网络 / NAT / 子网 / 最长前缀	Network
MAC / 相邻节点 / 帧 / ARP / Ethernet	Link
信号 / 光纤 / 电压 / 比特	Physical

3.4 Encapsulation (PPT 例子: DHCP)

发送时**自上而下**, 每层加 header:

Application data (DHCP message)	
↓ + UDP header	→ UDP segment
↓ + IP header	→ IP datagram
↓ + Ethernet header	→ Frame
↓ Physical	→ Bits

PPT 原文: “DHCP request **encapsulated** in UDP, encapsulated in IP, encapsulated in 802.3 Ethernet”

接收方反过来 **demultiplexing / demuxed**: Ethernet → IP → UDP → DHCP。

关键直觉: DHCP 本身只是 application layer 协议, 不”包含”多层; 是 DHCP 消息**经过**多层被包了多个 header (俄罗斯套娃)。

Header / Trailer 加在哪些层 (PPT 原话): – **Headers** are added to the data at layers **6, 5, 4, 3, 2** (用 OSI 编号; 对应 5 层模型中的 Application/Transport/Network/Link) – **Trailers** are usually added **only at layer 2** (Link 层, 例如 Ethernet 帧尾的 CRC 校验)

4. Web Request to www.google.com (综合流程)

顺序必背: DHCP → ARP → DNS → TCP → HTTP

Step 1: DHCP — 拿到上网身份

刚连上 WiFi 时, client 只有 MAC, 没有 IP / 网关 IP / DNS server IP。

DHCP 四步流程 (DORA):

Client	—	DHCP Discover (broadcast)	→	Server
Client	←	DHCP Offer	→	Server
Client	—	DHCP Request	→	Server
Client	←	DHCP ACK	→	Server

- 为何 **broadcast**: client 没 IP, 源 = 0.0.0.0、目的 = 255.255.255.255, 目标 MAC = FF:FF:FF:FF:FF:FF
- 为何用 **UDP**: 还没 IP, 没法做 TCP 握手
- **DHCP 在协议栈位置**: Application layer

Step 1 完成后 (PPT 原话): “Client now has IP address, knows name & addr of DNS server, IP address of its first-hop router”: – **✓** 自己的 IP – **✓**

subnet mask – **✓** first-hop router (gateway) 的 IP (不是 MAC!) – **✓** DNS server 的 IP – **✓** lease time (租期)

Step 2: ARP — 拿到网关 MAC

Client 知道网关 IP, 但 Ethernet frame 需要 MAC。

PPT 原话: ARP query broadcast, received by router, which replies with ARP reply giving MAC address of router interface.

重要直觉: – DHCP 不直接给 MAC: 分层设计——MAC 由 ARP 单独处理; MAC 易变 (换网卡), IP 配置稳定。 – **Gateway = first-hop router** (同义词)。

Step 3: DNS — 查 Google IP

DNS query → UDP → IP → Ethernet (目标 **MAC = 网关 MAC**) → 路由到 DNS server → 返回 **www.google.com** 的 IP。

PPT: IP datagram forwarded from campus network into comcast network, routed (tables created by **RIP, OSPF, IS-IS and/or BGP** routing protocols) to DNS server.

重要直觉: – **DNS server 不在你的子网**, 所以不需要它的 **MAC** – 你只把包交给 first-hop router, 由它一跳跳转过去 – 规则: **MAC 是”下一跳”的地址, IP 是”最终目的地”的地址**

Step 4: TCP 三次握手

Client	—	SYN	→	Server
Client	←	SYNACK	→	Server
Client	—	ACK + HTTP request	→	Server

← 第三次 ACK 可与 HTTP 一起发

1 RTT 完成连接建立 (client 视角)。

Step 5: HTTP

Client	—	GET / HTTP/1.1	→	Server
Client	←	HTTP response (网页内容)	→	Server

Step 5 用时 ≈ 1 RTT + 传输时间。

MAC 在整个流程中的使用

步骤	目标 MAC
DHCP Discover / Request	FF:FF:FF:FF:FF:FF (广播)
ARP Request	FF:FF:FF:FF:FF:FF (广播)
DNS Query / TCP / HTTP (一切发往外网)	网关的 MAC

ARP 只查 1 次, 之后所有出网包**复用网关 MAC**。每经过一个路由器, MAC 头被换一次 (每条 link 用各自的 MAC); **IP 头从头到尾不变** (始终指向 Google IP)。

5. Application Layer Principles

5.1 Network apps 跑在哪里

PPT 原话: write programs that **run on (different) end systems**, communicate over network. **No need to write software for network-core devices** — network-core devices do not run user applications.

意义: 在 end systems 上写 → 快速迭代部署, 不动核心路由器。

5.2 Client-server vs Peer-peer (PPT 直接引用)

维度	Client-server	Peer-peer (P2P)
Server	always-on host, permanent IP , often in data centers	no always-on server
Client / Peer	主动联系 server; intermittent; 可能 dynamic IP; clients 不直接互通	arbitrary end systems directly communicate
Scalability	server 易成瓶颈	self scalability — new peers bring new service capacity AND new service demands
管理	简单	complex (peers 可能 intermittent / 改 IP)
例子	HTTP, IMAP, FTP	P2P file sharing (BitTorrent), streaming (YouTube/Netflix), VoIP (Skype)

5.3 How processes communicate

- process = program running within a host
- 同一 host 内: **inter-process communication** (OS 提供)
 - 不同 host: 交换 **messages**

角色	定义
client process	initiates communication
server process	waits to be contacted

P2P 应用同时有 client process 和 server process (角色按“每次通信”动态分配)。

5.4 Sockets (PPT 原话)

socket analogous to **door**: an **interface**. – sending process pushes message out door – sending process relies on transport infrastructure on other side of door to deliver message to socket at receiving process – **two sockets involved: one on each side**

应用层在 socket 上方 (开发者控制), transport / network / link / physical 在 socket 下方 (OS 控制)。

5.5 Addressing processes (PPT 原话)

- to receive messages, process must have **identifier** identifier includes both **IP address** and **port numbers** associated with process on host.
- IP 标识 host (IPv4 = **32-bit**; IPv6 = **128-bit**)
 - Port 标识 host 上的进程

服务	默认端口
HTTP	80
HTTPS	443
Mail (SMTP)	25
DNS	53
DHCP server / client	67 / 68

IPv6 写法: 8 组、每组 4 位十六进制、: 分隔; 前导 0 可省; 连续零组用 :: 替代 (只能用一次)。

5.6 Transport service requirements (PPT 四类)

需求	含义	典型应用
data integrity	100% reliable (some apps tolerate loss)	file transfer, web, e-mail
timing	低延迟	Internet telephony, interactive games (10's msec)
throughput	最低带宽 (“elastic apps” 能用多少用多少)	streaming audio/video
security	加密、数据完整性	online banking

常见 apps 具体需求 (PPT 原话表, throughput 数值是考点):

application	data loss	throughput	time sensitive?
file transfer / download	no loss	elastic	no
e-mail	no loss	elastic	no
Web documents	no loss	elastic	no
real-time audio/video	loss-tolerant	audio 5Kbps–1Mbps ; video 10Kbps–5Mbps	yes, 10's msec
streaming audio/video	loss-tolerant	same as above	yes, few secs
interactive games	loss-tolerant	Kbps+	yes, 10's msec
text messaging	no loss	elastic	yes and no

5.7 TCP vs UDP (PPT 原话直接引用)

TCP service	UDP service
reliable transport between sending & receiving process	unreliable data transfer
flow control : sender won't overwhelm receiver	does not provide : reliability, flow control, congestion control, timing, throughput guarantee, security, or connection setup
congestion control : decelerate sender when network is overloaded	
connection-oriented : setup required between client and server processes	

应用与传输协议对应:

Application	App-layer protocol	Transport
file transfer	FTP	TCP
e-mail	SMTP	TCP
Web documents	HTTP 1.1	TCP
streaming audio/video	HTTP, DASH	TCP
Internet telephony	SIP, RTP	TCP or UDP
interactive games	proprietary	UDP or TCP
DNS	DNS	UDP

6. Web and HTTP

6.1 Web 基本概念 (PPT 原话)

web page consists of **objects**, each of which can be stored on different Web servers. Object can be HTML file, JPEG image, Java applet, audio file, ... web page consists of **base HTML-file** which includes **several referenced objects**, each addressable by a **URL**.

URL 例: **www.someschool.edu/someDept/pic.gif** = host name + path name。

6.2 HTTP Overview (PPT 原话)

HTTP: hypertext transfer protocol. Web's application-layer protocol. client/server model. – **client**: browser that requests, receives, (using HTTP protocol) and “displays” Web objects – **server**: Web server sends (using HTTP protocol) objects in response to requests

6.3 HTTP uses TCP / HTTP is “stateless”

1. client initiates TCP connection (creates socket) to server, port
2. server accepts TCP connection from client
3. HTTP messages exchanged between browser (HTTP client) and Web server
4. TCP connection closed

HTTP is “stateless”: server maintains **no information about past client requests**.

aside (PPT 原话): protocols that maintain “**state**” are complex! – past history (state) must be maintained – if server/client crashes, their views of “state” may be **inconsistent**

→ 两点都是 stateful 的坏处。HTTP 选择 stateless 是为了简单+易恢复; 登录/购物车等“状态”由 **cookies** 在外部补足。

6.4 HTTP Connections: Two Types

Non-persistent HTTP	Persistent HTTP (HTTP 1.1)
1. TCP connection opened	TCP connection opened to a server
2. at most one object sent over TCP connection	multiple objects can be sent over single TCP connection
3. TCP connection closed	TCP connection closed
downloading multiple objects requires multiple connections	Server keeps connection open after sending response

6.5 Non-persistent HTTP Response Time (PPT 原话)

RTT (definition): time for a message/small packet to travel from client to server and back.

HTTP response time (per object): 1. one RTT to **initiate TCP connection** 2. one RTT for **HTTP request and first few bytes of HTTP response** to return 3. **object/file transmission time**

Non-persistent HTTP response time per object = 2RTT + file transmiss

0.5 RTT ≈ 单向时间 (假设上下行对称, 课程默认)。

6.6 Non-persistent HTTP issues (PPT 原话)

- requires 2 RTTs per object
- OS overhead for **each** TCP connection
- browsers often open **multiple parallel TCP connections** to fetch referenced objects in parallel

6.7 Persistent HTTP 的两个 Option (PPT 原话)

Option 1: client sends requests **one by one** for the objects. (逐个: 发→等→发→等) **Option 2**: client sends requests **as soon as it encounters a referenced object**. (pipelining: 解析 HTML 时一遇到引用就立刻发请求, 不等之前的 response)

效果： – Option 1: 每个 object 1 RTT, N 个 object $\approx$ N RTT – Option 2 (pipelining): 所有 request 几乎同时出发, 全部 $\approx$ 1 RTT

6.8 RTT 计数总结 (综合三种加速手段)

模式	第 1 个 object	后续 object	N object 总用时
Non-persistent	2 RTT + 传输	2 RTT + 传输 (重新握手)	$N \times (2RTT + \text{传输})$
Persistent (Option 1)	2 RTT + 传输	1 RTT + 传输	$2RTT + (N-1) \times 1RTT + N \times \text{传输}$
Persistent + pipelining (Option 2)	2 RTT + 传输	共享 1 RTT	$\approx 2RTT + N \times \text{传输}$
浏览器并行 6 条 TCP	—	—	总用时 $\approx$ 串行 / 6

实际浏览器三种全用。

6.9 HTTP Request / Response Message (基本格式)

Request:

```
GET /path HTTP/1.1\r\n
Host: www-net.cs.umass.edu\r\n
User-Agent: Mozilla/5.0 ...\r\n
Connection: keep-alive\r\n
\r\n
```

- request line (GET / POST / HEAD) + header lines + 空行 + body
- ASCII 格式

Response:

```
HTTP/1.1 200 OK\r\n
Date: ...
Server: Apache/...
Content-Length: 2651
Content-Type: text/html
\r\n
data data data ...
```

- status line (version + 状态码 + phrase) + header lines + 空行 + body

常见状态码: 200 OK, 301 Moved Permanently, 400 Bad Request, 404 Not Found, 505 HTTP Version Not Supported.

7. Cookies (HTTP 状态维持)

7.1 机制 (PPT 原话)

Web sites and client browser use cookies to maintain some state between transactions.

四步：1. 首次访问, server 创建 unique ID (cookie) + backend database 中的 entry 2. response 带 Set-cookie: 1678 → 浏览器把 cookie 存入本地 cookie file 3. 之后请求自动带 cookie: 1678 4. server 用 ID 查 backend database 识别用户, 做 cookie-specific action

7.2 What cookies can be used for

authorization、shopping carts、recommendations、etc.

7.3 Cookies and Privacy (PPT 原话)

Cookies can be used to: – track user behavior on a given website (first party cookies) – track user behavior across multiple websites (third party cookies) without user ever choosing to visit tracker site (!) – tracking may be invisible to user.

机制：访问 nytimes.com → 页面嵌入 AdX.com 广告 → 浏览器也向 AdX.com 发请求 → AdX.com set 自己的 cookie。之后访问 socks.com 若也嵌 AdX.com, AdX 通过同一个 cookie 跨站追踪你的浏览行为。

third party tracking via cookies: disabled by default in Firefox, Safari browsers.

类型	来源
First-party cookie	你主动选择访问的网站设置
Third-party cookie	嵌入页面的第三方 (你从未选择访问) 设置

8. Web Caches (Proxy Servers)

8.1 定义与原理 (PPT 原话)

Goal: satisfy client requests without involving origin server. – if object in cache: cache returns object to client – else cache requests object from origin server, caches received object, then returns object to client

8.2 Cache 的双重角色 (PPT 原话)

Web cache acts as both client and server: – server for original requesting client – client to origin server

8.3 Why Web caching? (PPT 原话)

- reduce response time for client request — cache is closer to client
- reduce traffic on an institution’s access link (瓶颈)
- Internet is dense with caches — enables “poor” content providers to more effectively deliver content

8.4 Caching Example (PPT Scenario, 必背)

参数	值
access link rate	154 Mbps
RTT from institutional router to server	~ 2 sec
web object size	100K bits
average request rate from browsers	15/sec
avg data rate to browsers	1.50 Mbps
LAN	1 Gbps

8.5 三段 delay

end-end delay = LAN delay + access link delay + Internet delay

段	值 (无 cache, 0.97 利用率)
LAN delay	毫秒级 (忽略)
access link delay	分钟级 (瓶颈! 高利用率→排队爆炸)
Internet delay	~ 2.0 sec
总 end-end delay	~ 2.0 sec + minutes

关键澄清： – “RTT from institutional router to server $\approx$ 2 sec” = Internet delay (机构路由器穿过公网到 origin server 的来回) – 这 2 秒已打包了 TCP+HTTP+传输+路由处理 (Web Cache 题不再拆 RTT) – access link delay 单独算 (看 utilization)

8.6 三个解决方案对比

Option	Access link rate	utilization	end-end delay	cost
不动	154 Mbps	0.97	~2 sec + minutes	—
Option 1: 升级链路	154 Mbps	0.0097	~2 sec + msecs	expensive!
Option 2: 装 Web cache	1.54 Mbps	0.58	~ 1.2 secs	cheap!

8.7 Calculation with cache (PPT 原话, hit rate = 0.4)

suppose cache hit rate is 0.4: – 40% requests served by cache, with low (msec) delay – 60% requests satisfied at origin servers – rate to browsers over access link = $0.6 * 1.50 \text{ Mbps} = 0.9 \text{ Mbps}$ – access link utilization = $0.9 / 1.54 = 0.58$ (previously 0.97) – average end-end delay = $0.6 * (\text{delay from origin servers}) + 0.4 * (\text{delay when satisfied at cache}) = 0.6 * 2.0 + 0.4 * (\sim \text{msecs}) \approx 1.2 \text{ secs}$
lower average end-end delay than with 154 Mbps link (and cheaper too!)

8.8 计算公式 (考试模板)

new access-link traffic = $(1 - \text{hit_rate}) \times \text{original_data_rate}$
new access link utilization = $\text{new_traffic} / \text{access_link_rate}$
average end-end delay = $(1 - \text{hit_rate}) \times \text{origin_delay} + \text{hit_rate} \times c$

9. E-mail / SMTP

9.1 三大组件 (PPT 原话)

Three major components: – user agents (a.k.a. “mail reader”) — composing, editing, reading mail messages, e.g., Outlook, iPhone mail client. Outgoing/incoming messages stored on server. – mail servers — mailbox contains incoming messages for user; message queue of outgoing (to be sent) mail messages. – simple mail transfer protocol: SMTP — between mail servers to send email messages. client = sending mail server, “server” = receiving mail server.

9.2 Alice → Bob 流程

Alice 写邮件 (UA)
↓
Alice 的 mail server (放入 message queue)
↓ SMTP over TCP
Bob 的 mail server (放入 mailbox)
↓
Bob 用 UA 读 (经 IMAP / POP3)

协议	作用
SMTP	mail server ↔ mail server, push/send
IMAP	UA ← mail server, pull/read

10. DNS (Domain Name System)

10.1 What is DNS (PPT 原话)

Domain Name System (DNS): – distributed database implemented in hierarchy of many name servers – application-layer protocol: hosts, DNS servers communicate to resolve names (address/name translation) – note: core Internet function, implemented as application-layer protocol

考点：DNS 虽然功能核心, 但属于应用层 (运行在 UDP 上, 端口 53)。

10.2 DNS Services

- hostname-to-IP-address translation

- **load distribution**: replicated Web servers — many IP addresses correspond to one name
- host aliasing (CNAME)
- mail server aliasing (MX)

10.2b Thinking about the DNS (PPT 原话, DNS 特征)

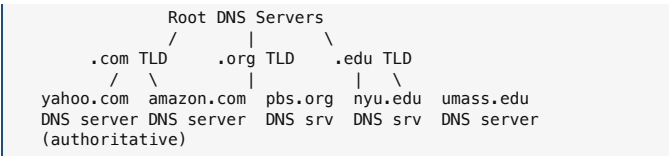
- **Very large distributed database**: ~ billion records, each simple
- handles **trillions of queries/day**: **many more reads than writes**
- **performance matters**: almost every Internet transaction interacts with DNS — **msecs count!**
- **organizationally, physically decentralized**: millions of different organizations responsible for their records
- 关键非功能需求: **reliability, security**

10.3 Why not centralize DNS? (PPT 原话)

原因	解释
single point of failure	一挂全 Internet 解析瘫痪
traffic volume	Comcast 一家 600B queries/day; Akamai 2.2T queries/day
distant centralized database	远距离查询 delay 高
maintenance	全球域名持续变化, 集中维护不现实

A: **doesn't scale!**

10.4 DNS Hierarchy (PPT 原话)



Root DNS servers (PPT 原话)

- *incredibly important* Internet function — Internet couldn't function without it!
- **DNSSEC** — provides security (authentication, message integrity)
- **ICANN** (Internet Corporation for Assigned Names and Numbers) manages root DNS domain
- **13 logical root name “servers”** worldwide; each “server” replicated many times (~200 servers in US)

TLD servers

- 负责 .com, .org, .net, .edu, .aero, .jobs, .museums, 国家域名 .cn/.uk/.fr/.ca/.jp 等
- Network Solutions: authoritative registry for **.com, .net** TLD
- Educause: **.edu** TLD

Authoritative DNS servers

- organization's own DNS server(s), providing authoritative hostname → IP mappings
- can be maintained by organization or service provider

10.5 Local DNS Server (PPT 原话)

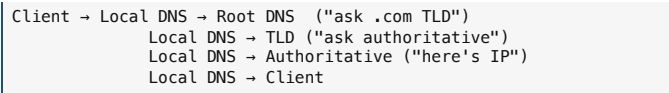
when host makes DNS query, it is sent to its **local DNS server** Local DNS server returns reply, answering: — from its **local cache** of recent name-to-address translation pairs (**possibly out of date!**) — forwarding request into DNS hierarchy for resolution — each ISP has local DNS name server

查找命令: MacOS `scutil --dns`; Windows `ipconfig /all`.

10.6 Iterated vs Recursive Query

Iterated query (PPT 原话)

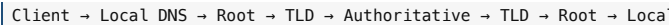
contacted server replies with name of **server to contact**. Like: “I don't know this name, but ask this server.”



- 负担主要在 **local DNS server**

Recursive query (PPT 原话)

puts **burden of name resolution on contacted name server**. heavy load at upper levels of hierarchy?



- 上层 DNS 负担重

10.7 DNS Caching (PPT 原话)

once (any) name server learns mapping, it **caches** mapping, and **immediately** returns a cached mapping in response to a query — caching improves response time — cache entries **timeout** (disappear) after some time (**TTL**) — TLD servers typically cached in local name servers
cached entries may be **out-of-date**: — if named host changes IP address, may not be known Internet-wide until all TTLs expire! — **best-effort name-to-address translation!**

10.8 DNS Records (必考)

DNS: distributed database storing **resource records (RR)** RR format: (**name, value, type, ttl**)

name 和 value 的具体含义取决于 **type**; ttl = 缓存有效期 (秒), 过期后必须重新查询。

type	name	value	例子
A (Address)	hostname	IP address	(www.example.com, 1.2.3.4, A)
NS (Name Server)	domain	hostname of authoritative name server for this domain	(example.com, dns.example.com, NS)
CNAME (Canonical Name)	alias	canonical (real) name	(www.ibm.com, servereast.backup2.ibm.com, CNAME)
MX (Mail eXchange)	domain	name of SMTP mail server associated with name	(example.com, mail.example.com, MX)

10.9 DNS Protocol Messages (PPT, basic understanding only)

DNS query 和 reply 用同一种 **message format**:

字段	长度/作用
identification	16 bit query ID (reply 用相同编号回应)
flags	query/reply、recursion desired、recursion available、reply is authoritative
# questions / # answer RRs / # authority RRs / # additional RRs	各部分计数
questions	name + type fields
answers	RRs in response to query
authority	records for authoritative servers
additional info	额外“helpful”RRs (如 NS 记录的 glue A 记录)

10.10 Getting Your Info into the DNS (PPT 例子: Network Utopia)

新公司 **networkutopia.com** 上线 DNS:

Step 1: 在 **DNS Registrar** (如 GoDaddy) 注册域名, 提供 authoritative name server (primary + secondary) 的 names 和 IPs。

Step 2: Registrar 把这两条 RR 插入 **.com TLD server**:

(networkutopia.com, dns1.networkutopia.com, NS) ← 告诉别人去问
(dns1.networkutopia.com, 212.212.212.1, A) ← glue record,

Step 3: 你在自己的 authoritative server (212.212.212.1) 上配置: — type A record for **www.networkutopia.com** — type MX record for **networkutopia.com**

外人怎么找到你: local DNS → root → .com TLD (拿到 NS+glue) → 你的 authoritative DNS → 拿到 **www.networkutopia.com** 的 IP。

10.11 DNS Security (PPT 原话)

DDoS attacks

- **attack root servers with traffic**
 - not successful to date
 - traffic filtering
 - local DNS servers cache IPs of TLD servers, allowing **root server bypass**
 - **attack TLD servers** — potentially more dangerous
- Spoofting
- **intercept DNS queries, returning fake replies** (DNS cache poisoning)
 - 用户输入正确域名, 但被引导到错误 IP
 - 防御: **DNSSEC** (数字签名验证 reply 真实性 + 完整性)

11. P2P File Distribution

11.1 P2P Architecture 核心特征 (PPT 原话, §5.2 已展开, 此处不重复)

- **no always-on server**, arbitrary end systems directly communicate
- peers request service from other peers, provide service in return
- **self scalability** — new peers bring new service capacity AND new service demands
- intermittently connected, change IP addresses → complex management
- 例子: **BitTorrent, YouTube/Netflix streaming, Skype VoIP**

实际运行机制: 文件切成 **chunks** → peer —拿到 chunk 立刻可以转发给别人 → 用 **bitmap** 互通“我有哪些 chunks” → **tit-for-tat** 优先把 chunk 给“上传给我最快”的 peer (防 free-rider) → 每 chunk 有 hash 防伪。Tracker / DHT 帮 peer 互相发现, 但不传内容。

11.2 File Distribution Time 模型 (PPT 必考)

问题：从 1 个 server 把 size F 的文件分发给 N 个 peers 需要多少时间？
网络拓扑：1 server + N peers = N+1 个设备；网络中间假设 abundant bandwidth，瓶颈在每个端的 upload/download capacity。

符号	含义
F	文件大小 (bits)
N	peer 数量
us	server upload capacity
ui	peer i 的 upload capacity
di	peer i 的 download capacity
dmin	最慢 peer 的 download capacity

11.3 Client-Server 公式 (PPT 原话)

server transmission: usually sequentially send (upload) N file copies – time to send one copy: F/us – time to send N copies: NF/us
client: each client must download file copy – $dmin = \min \text{ client download rate} - \min \text{ client download time: } F/dmin$

$Dc-s \geq \max \{ NF/us , F/dmin \}$

increases linearly in N

误区澄清：“logical 上 server 只存一份文件”≠“网络只传一次”。每个 client 各有自己的连接，server 网卡必须把 F bits 推 N 次 → NF/us 。只有 multicast/CDN 才能“少发几次”。

11.4 P2P 公式 (PPT 原话)

server transmission: upload at least one copy – time to send one copy: F/us
client: each client must download file copy – $\min \text{ client download time: } F/dmin$
clients: as a union, download NF bits – max upload rate (will limiting max download rate) is $us + \sum ui$

$DP2P \geq \max \{ F/us , F/dmin , NF/(us + \sum ui) \}$

increases linearly in N ... but so does this, as each peer brings service capacity

11.5 为什么是 max 而不是相加 (核心直觉)

P2P 不是”seeder 发完整文件 → 然后 peers 互传”的串行流程。chunks 边收边发：seeder 才发出 chunk 1 给 peer A，peer A 立刻可以把 chunk 1 转给 peer B，同时 seeder 继续发 chunk 2 给 peer C。
三个约束并行同时发生，所以取最慢的那个：

约束	物理意义
F/us	seeder 至少要把完整 F 送出去一次（让内容进入网络）
$F/dmin$	最慢 peer 至少要完整收一次（瓶颈接收方）
$NF/(us+\sum ui)$	全网共需流通 NF bits，总上传容量 = $us+\sum ui$

P2P 里的“server” = initial seeder (拥有完整文件的初始 peer)，不是 always-on 专职 server。

11.6 C-S vs P2P 增长趋势

- $Dc-s = NF/us$: 随 N 线性增长 (server 一人扛)
- $DP2P = NF/(us+\sum ui)$: N 越大分母同步增大，几乎不增长 (peer 共担)

PPT 图示：N 增大时 P2P 曲线远低于 C-S 线性曲线。

12. Video Streaming + DASH + CDN

12.1 Context (PPT 原话)

stream video traffic: major consumer of Internet bandwidth – Netflix, YouTube, Amazon Prime: 80% of residential ISP traffic (2020)
challenges: – scale — how to reach ~1B users? – heterogeneity — different users have different capabilities (wired vs mobile, bandwidth rich vs poor)
solution: distributed, application-level infrastructure

12.2 Video 编码

video: sequence of images displayed at constant rate (e.g., 24–60 images/sec)
digital image: array of pixels, each represented by bits coding: use redundancy within and between images to decrease # bits – spatial (within image) – temporal (from one image to next)

类型	含义
CBR (Constant Bit Rate)	video encoding rate fixed
VBR (Variable Bit Rate)	encoding rate changes with spatial/temporal coding

例子：MPEG1 (CD-ROM) 1.5 Mbps; MPEG2 (DVD) 3–6 Mbps; MPEG4 (Internet) 64Kbps – 12 Mbps。

计算练手：32x32 RGB image，每通道 8 bits → $32 \times 32 \times 3 \times 8 = 24576 \text{ bits}$ 。

12.3 Streaming Stored Video

简单场景：video server → Internet → client。

主要挑战：– server-to-client bandwidth vary over time (congestion 变化) – packet loss, delay due to congestion → delay playout 或 poor quality – continuous playout constraint: playout timing 必须匹配原始 timing – network delays are variable → 需要 client-side buffer 来吸收 jitter – 还要支持 pause / fast-forward / rewind / 重传丢包

12.4 Playout Buffering (PPT 原话)

client-side buffering and playout delay: compensate for network-added delay, delay jitter

server CBR transmission → 网络 variable delay → client 收到后先 buffer → constant rate 播放。

代价：buffer 太小→卡顿；太大→启动等待长。

12.5 DASH (Dynamic Adaptive Streaming over HTTP, PPT 原话)

Server: – divides video file into multiple chunks – each chunk encoded at multiple different rates – different rate encodings stored in different files – files replicated in various CDN nodes – manifest file: provides URLs for different chunks

Client: – periodically estimates server-to-client bandwidth – consulting manifest, requests one chunk at a time – chooses maximum coding rate sustainable given current bandwidth – can choose different coding rates at different points in time, and from different servers

“intelligence” at client (决定三件事) : – “when” to request chunk (避免 buffer hunger / overflow) – “what” encoding rate (带宽多就高质量) – “where” to request from (选 close 或 high bandwidth 的 URL server)

Streaming video = encoding + DASH + playout buffering

12.6 CDN (Content Distribution Network, PPT 原话)

Challenge: how to stream content (selected from millions of videos) to massive users, concurrently?

Option 1: single, large “mega-server” – single point of failure – point of network congestion – long (and possibly congested) path to distant clients – doesn’t scale!

Option 2: store/serve multiple copies of videos at multiple geographically distributed sites (CDN) – enter deep: push CDN servers deep into many access networks → close to users – e.g., Akamai: 240,000 servers deployed in > 120 countries (2015)

12.7 DASH 与 CDN 的关系

- DASH 解决: what bitrate should I download?
- CDN 解决: where should I download it from?
- 结合: client 根据当前带宽选 bitrate，从最近的 CDN node 下载相应 chunk。

13. 速查表

13.1 Layer Matching

Protocol	Layer
HTTP, DNS, SMTP, IMAP, FTP, DHCP	Application
TCP, UDP	Transport
IP, ICMP, BGP, OSPF, RIP, IS-IS	Network
Ethernet (802.3), WiFi (802.11), ARP	Link

13.2 数值速查

项	值
IPv4 长度	32 bit (4 段十进制)
IPv6 长度	128 bit (8 组 4 位十六进制)
HTTP 端口	80
HTTPS 端口	443
SMTP 端口	25
DNS 端口	53 (UDP)
DHCP 客户端口 / 服务端口	68 / 67
13 logical root name “servers”	~200 servers in US
Light speed in fiber	200,000 km/s
n 设备直连线数	$n(n-1)/2 = O(n^2)$
Non-persistent HTTP per object	2RTT + transmission time
32x32 RGB (8 bit/通道) 大小	$32 \times 32 \times 3 \times 8 = 24576 \text{ bits}$
Client-server 分发时间下界	$Dc-s \geq \max\{NF/us, F/dmin\}$
P2P 分发时间下界	$DP2P \geq \max\{F/us, F/dmin, NF/(us+\sum ui)\}$
Web cache 平均 delay	$(1-h) \cdot D_origin + h \cdot D_cache$
Web cache 新流量	$(1-h) \cdot \text{原 traffic}$
视频流量占比 (2020)	Netflix+YouTube+Prime ≈ 80% ISP traffic
Akamai 规模 (2015)	240,000 servers / 120+ countries

13.3 易混淆对照

混淆点	区别
Host vs Client/Server	Host 是设备；Client/Server 是角色
Gateway vs First-hop router	同义词
DHCP 给 IP, ARP 给 MAC	DHCP 给 IP/网关 IP/DNS IP; ARP 给网关 MAC

混淆点	区别
DNS vs ARP	DNS: hostname → IP; ARP: IP → MAC (同子网内)
MAC vs IP	MAC = “下一跳”地址 (每跳更换); IP = “最终目的地”地址 (全程不变)
Iterated vs Recursive DNS	Iterated: 被问者返回“下一站”;负担在 local. Recursive: 被问者继续追问; 负担在上层。
A / NS / CNAME / MX	域名→IP / 域名→DNS server / 别名→真名 / 域名→邮件 server
First-party cookie vs Third-party	主动访问的网站 vs 嵌入广告的第三方
2 sec Internet delay vs access link delay	2 sec 是 Internet 段 (稳定值); access link delay 看 utilization
Persistent vs Pipelining	持久 = 复用 TCP; pipelining = 不等 response 连续发 request
Client-server vs P2P 分发	C-S: server 一人扛 N 份 (NF/us); P2P: 所有 peer 共担 (NF/(us+Σui))
DASH vs CDN	DASH 决定 what bitrate; CDN 决定 where to get
CBR vs VBR	编码速率固定 vs 跟内容复杂度变化

13.4 核心公式速查

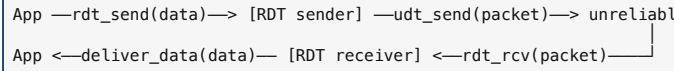
非持久 HTTP per object = 2 RTT + transmission
Web cache new traffic = (1-h) × original
Web cache avg delay = (1-h) × D_{origin} + h × D_{cache}
Client-server 分发 Dc-s ≥ max{NF/us, F/dmin}
P2P 分发 DP2P ≥ max{F/us, F/dmin, NF/(us+Σui)}
RGB image bits = pixels × 3 × bits_per_channel

COMP5311 Lecture 4 + 5 Cheat Sheet — Reliable Data Transfer & TCP

对应课件: Lecture 4 (RDT) slides 5–44; Lecture 5 (TCP) slides 4–75 范围:
rdt1.0→3.0, GBN/SR, TCP segment, seq/ACK, RTT/timeout, 重传、Fast Retransmit, Flow Control, 3-way Handshake, Congestion Control, AIMD, Slow Start, ssthresh, Tahoe/Reno, Delay-based (BBR), Fairness

1. Reliable Data Transfer 演进

1.1 模型与四个函数



- rdt = reliable, udt = unreliable
- 单向数据传输, 但 ACK/NAK 控制信息双向流动
- 协议用 FSM (Finite State Machine) 描述: 当前 state + event → action + 下一个 state

1.2 协议升级表 (“问题→机制”)

版本	信道假设	新增机制	解决的核心问题
rdt1.0	完美信道	无	—
rdt2.0	比特错误	checksum + ACK/NAK + retransmission	检测错误并恢复
rdt2.1	ACK/NAK 也可能坏	sequence number (0/1)	区分新包 vs 重传重包
rdt2.2	同 2.1, 去掉 NAK	ACK 只用, ACK 必须带 seq #; duplicate ACK = NAK	简化协议 (TCP 风格)
rdt3.0	增加丢包	countdown timer + timeout 重传	ACK/包丢失时不会死等

1.3 Checksum (rdt2.0)

PPT 算法: 1. 把数据分段求和; 2. 取反码 (one’s complement) = checksum; 3. 发送 data + checksum; 4. receiver 把所有数 (含 checksum) 相加, 结果应全为 1, 否则有错。

数学事实: x + ~x 在每一位都是“一个 0 一个 1”, 普通二进制加法即得全 1 且无进位——这跟“反码/补码”表示无关。补码会再 +1 才归 0; 反码停在全 1。

1.4 rdt2.0 的 fatal flaw

fatal flaw = 致命缺陷。如果 ACK/NAK 自己坏了, sender 不知道 receiver 的状态: - 重传 → 可能 duplicate - 不重传 → 可能丢失 没有序号, receiver 无法区分“新包”和“重传的旧包”, 所以协议无解。

→ rdt2.1 加 0/1 序号修复。stop-and-wait 一次只有一个未 ACK 包, 所以两个序号交替即可。

1.5 rdt3.0 四种场景

场景	描述	关键行为
(a) no loss	正常	send → ACK → next

场景	描述	关键行为
(b) packet loss	pkt 丢	timeout → 重传
(c) ACK loss	pkt 到, ACK 丢	timeout → 重传 → receiver 用 seq # 识别 duplicate → 重发 ACK
(d) premature timeout	pkt/ACK 只是延迟	误以为丢, 重传; 后到的 delayed ACK 时 sender ignore (已推进)

性能瓶颈: stop-and-wait——发一个等一个, 链路利用率极低 (U = (L/R) / (RTT + L/R))。

2. Pipelined Protocols: GBN vs SR

2.1 Pipelining 思想

sender allows multiple, “in-flight”, yet-to-be-acknowledged pkts

要求: ① seq # 空间扩大 (k bit → 2^k 个编号); ② sender/receiver buffering。

2.2 GBN vs SR 对比表

特性	Go-Back-N	Selective Repeat
sender window	≤ N 个连续未 ACK	≤ N 个未 ACK
ACK 类型	cumulative ACK ACK(n) 表示 ≤n 全收	individual ACK, 逐个确认
receiver 缓存失序包	✗ 丢弃	✓ 缓存
timer	1 个 (给 oldest in-flight)	每个未 ACK 包一个
超时重传	重传 window 内所有 ≥n 的包	只重传超时那一个
receiver 复杂度	简单	复杂
丢包时浪费	高	低
GBN window 上限	2 ^k - 1	≤ 2 ^k / 2

2.3 GBN 行为示例 (N=4, pkt2 丢失)

sender: send 0,1,2,3
receiver: pkt0 ✓→ACK0; pkt1 ✓→ACK1; pkt2 X; pkt3-discard,(re)send ACK
sender 收 ACK1 后继续发 pkt4,pkt5 → receiver 全部 discard, (re)send ACK
pkt2 timeout → sender 重传 pkt2,pkt3,pkt4,pkt5

名字由来: 超时后“go back N”, 从丢失处全部重发。

2.4 SR 行为示例 (同样 pkt2 丢失)

receiver: pkt0,1 ✓; pkt2 X; pkt3,4,5 缓存,各发 ACK3/4/5
sender: 收 ACK3/4/5 标记已确认, 但 send_base 卡在 2
pkt2 timeout → 只重传 pkt2
pkt2 ✓ → ACK2, receiver 一次性 deliver pkt2,3,4,5

2.5 SR receiver 完整窗口逻辑

收到 pkt n 落在	行为
[rcvbase, rcvbase+N-1]	send ACK(n); 失序则缓存; 按序则 deliver (带上已缓存的连续部分), 窗口前移
[rcvbase-N, rcvbase-1] (旧窗口)	仍然要发 ACK(n)
其他	ignore

为什么旧窗口的包要重 ACK: sender 上次的 ACK 可能丢了, 正在超时重传; 不重 ACK 会导致 sender 死循环。

2.6 接收 vs 交付

- 接收: 失序也接, 缓存
- 交付给应用层: 必须按序 (应用期待有序字节流)

3. TCP Overview (7 大特性)

特性	说明
point-to-point	一对一, 无广播
reliable, in-order byte stream	字节流, 无消息边界
full duplex	双向同时传
MSS	Maximum Segment Size, 应用层数据上限 (典型 1460B = MTU 1500 - IP20 - TCP20)
cumulative ACKs	ACK n 表示 n 之前全收
pipelining	窗口大小由 cwnd 与 rwnd 共同决定
connection-oriented	三次握手初始化状态
flow controlled	sender 不会压垮 receiver

4. TCP Segment、Sequence Number、ACK

4.1 关键字段

字段	含义
sequence number (32-bit)	本 segment 数据中第一个字节在字节流中的编号
ACK number	下一个期待收到的字节编号 (cumulative)
receive window (rwnd)	流量控制, receiver buffer 剩余
C/E	拥塞标志 (ECN)
RST/SYN/FIN	连接管理
checksum	Internet checksum

4.2 Sequence/ACK 规则

- TCP 是字节级编号, 不是包编号
- ACK = 对方 Seq + 数据字节数 → 下一个期待字节
- SYN 和 FIN 各占 1 个序号 (虚拟字节), 所以握手时 ACK = x+1, y+1
- TCP 全双工: 两条字节流各自独立编号, A→B 用 ISN_A, B→A 用 ISN_B, 互不相干。同一 segment 里的 Seq 与 ACK 指两条不同的流
- ISN 随机 (防旧连接残留 + 防攻击); 32-bit 字段会回绕, 高速链路用 PAWS (timestamp) 区分

4.3 Slide 48 例子

A → B: Seq=42, ACK=79, data='C' (1 字节)
B → A: Seq=79, ACK=43, data='C' (echo, piggyback ACK)
A → B: Seq=43, ACK=80

5. RTT 估算与 TimeoutInterval

5.1 三个公式 (迭代更新; 左侧 = 新值, 右侧 = 旧值)

$$\text{EstimatedRTT}_{\text{new}} = (1 - \alpha) \cdot \text{EstimatedRTT}_{\text{old}} + \alpha \cdot \text{SampleRTT}, \quad \alpha = 0.125$$

$$\text{DevRTT}_{\text{new}} = (1 - \beta) \cdot \text{DevRTT}_{\text{old}} + \beta \cdot |\text{SampleRTT} - \text{EstimatedRTT}_{\text{old}}|, \quad \beta = 0.75$$

$$\text{TimeoutInterval} = \text{EstimatedRTT} + 4 \cdot \text{DevRTT}$$

- EWMA: 旧样本影响指数衰减, 越新权重越大
- DevRTT 是“波动幅度”, 给 timeout 加 safety margin (x4)
- 计算顺序: 先用旧 EstimatedRTT 算 DevRTT, 再更新 EstimatedRTT, 最后算 TimeoutInterval
- SampleRTT 测量时忽略重传

5.2 Timeout 的权衡

太短	太长
premature timeout, 多余重传, 浪费带宽, 加剧拥塞	反应迟缓, 丢包发现慢

TimeoutInterval 的用途: sender 计时器的超时阈值; 超时未收 ACK → 判定丢失 → 触发重传。

6. TCP 重传场景与 Fast Retransmit

6.1 三种 Retransmission 场景

场景	行为	结果
(a) lost ACK	Seq=92 发出, ACK=100 丢失 → timeout → 重传 Seq=92	receiver 重发 ACK=100
(b) premature timeout	92 和 100 都发出; ACK=100 迟到; timeout → 重传 92 → receiver 回 cumulative ACK=120	sender 多发一次, 但 cumulative ACK 推进正确
(c) cumulative ACK 覆盖	ACK=100 丢, ACK=120 到达	无需重传, 120 已确认 100 之前全部

6.2 Fast Retransmit (slide 34)

收到 3 个 duplicate ACK (同一 ACK 号共 4 次) → 立刻重传“smallest seq # 的未 ACK 包”, 不等 timeout。

为什么 3 个: 网络可能乱序, 1~2 个 dup 可能只是 reorder; 3 个说明已有 3 个后续 segment 到达 receiver, 缺的那个高概率真丢了。

只重发缺的那一个: 3、4、5 已经被 receiver 缓存 (正是它们引发 dup ACK), 只需补 #2, receiver 收到后用 cumulative ACK 一次性确认整段 (如 ACK=6)。

机制	触发	速度
timeout retransmission	timer 到期	慢
fast retransmit	3 dup ACKs	快

6.3 重复 ACK 的产生机制

receiver 收到失序包时, 不更新 ACK 号, 重发上一个 cumulative ACK:

sender 发 1,2,3,4,5
#1 ✓ → ACK 2
#2 丢; #3 到 → ACK 2 (dup 1)
#4 到 → ACK 2 (dup 2)
#5 到 → ACK 2 (dup 3) → 触发 fast retransmit

7. TCP Flow Control

7.1 问题

如果 network layer 把数据放进 receive buffer 的速度 > application read() 的速度, buffer 会溢出。

7.2 机制

- TCP receiver 在 header 的 rwnd (receive window) 字段通告剩余 buffer 空间
- sender 限制: LastByteSent - LastByteAcked ≤ rwnd
- 保证 receive buffer 不溢出
- RcvBuffer 默认 4096 bytes (典型 OS 自动调整)

7.3 Sender 实际窗口

$$\text{sender window} = \min(\text{rwnd}, \text{cwnd})$$

- rwnd 保护 receiver buffer
- cwnd 保护 network
- 谁更紧用谁

7.4 Flow vs Congestion Control

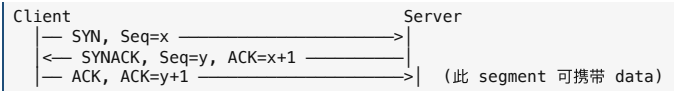
	Flow Control	Congestion Control
保护	receiver buffer	network
控制变量	rwnd	cwnd
反馈	receiver 明告	sender 推断 (loss/delay) 或 router feedback
一句话	don't overwhelm the receiver	don't overwhelm the network

8. TCP Connection Management

8.1 为什么不能用 2-way handshake

PPT 给出两个失败场景: 1. half-open: 旧 SYN 延迟到达, client 已关闭。server 误以为新连接 → 浪费资源、无对应 client 2. duplicate data: 旧 data 在新连接建立后到达, 被当作合法数据接收
→ 必须用 3-way handshake + 随机 ISN 防御

8.2 三次握手流程



步骤	含义
SYN	client 请求, 告知 ISN_x
SYNACK	server 同意, 告知 ISN_y, 同时确认 client 的 SYN
ACK	client 确认 server 的 SYN

+1 的原因: SYN 标志位本身占 1 个序号 (虚拟字节), 所以下一个真实数据从 x+1 开始; FIN 同理。

8.3 状态机

角色	状态转换
Client	LISTEN → SYNSENT → ESTAB
Server	LISTEN → SYN_RCVD → ESTAB

8.4 各步骤之后双方知道什么 (PPT 高频考点)

阶段	Client 知道	Server 知道
1st 后	nothing	client can send & server can receive
2nd 后	client send/receive & server send/receive	同 1st (仍不确定 client 能否 receive)
3rd 后	全部 send/receive	全部 send/receive

第 2 次握手后 server 还不知道 client 能否接收——所以需要第 3 次。

9. Principles of Congestion Control

9.1 Congestion 定义

“too many sources sending too much data, too fast for network to handle”

表现: 长排队延迟、丢包、重传、带宽浪费、throughput 下降。

9.2 三个 Scenario (递进揭示拥塞代价)

Scenario	假设	揭示的代价
1: 1 router, infinite buffers	不丢包	λ_in 接近 R/2 时, delay → ∞
2: 1 router, finite buffers	会丢包	(i) 必要重传 (包真丢) (ii) un-needed duplicates: premature timeout 引起多发副本 → 进一步降低 effective throughput

Scenario	假设	揭示的代价
3: multi-hop paths + retransmit	包在下游被 drop	上游链路 + buffer 全部白用；负载过高时 throughput → 0

9.3 PPT Insights (slide 62) — 5 条标准答案

- throughput **never exceeds capacity**
- delay **increases as capacity approached**
- loss/retransmission **decreases** effective throughput
- un-needed duplicates** further decreases effective throughput
- upstream transmission/buffering **wasted** for packets lost downstream

9.4 Approaches

方法	说明	例子
End-end	网络无显式反馈, sender 从 loss/delay 推断	TCP (主流)
Network-assisted	路由器直接告知拥塞	ECN

10. TCP Congestion Control

10.1 cwnd (congestion window)

$$\text{LastByteSent} - \text{LastByteAcked} \leq \text{cwnd}$$

- in-flight 字节数上限, sender 自行维护
- TCP rate $\approx \frac{\text{cwnd}}{\text{RTT}}$ bytes/sec
- 动态根据网络状况调整

10.2 AIMD (Additive Increase, Multiplicative Decrease)

阶段	行为
Additive Increase	每 RTT 增加 1 MSS (线性)
Multiplicative Decrease	loss 发生 → cwnd 减半

不对称设计: 温和涨、果断退。在共享瓶颈时 AIMD 会收敛到公平 (多流自动均分)。

PPT 论点 (slide 67): AIMD 是 **distributed + asynchronous** 算法, 能 **optimize congested flow rates network-wide**, 具 **stability properties**。

10.3 Slow Start

初始 cwnd = 1 MSS 每收到一个 ACK → cwnd += 1 MSS ⇒ 每 RTT 翻倍 (指数增长)
--

名为 slow, 实则增长很快。RTT 0:1 → 1:2 → 2:4 → 3:8 → ...

10.4 ssthresh & 切换到 Congestion Avoidance

条件	行为
cwnd < ssthresh	Slow Start (指数)
cwnd ≥ ssthresh	Congestion Avoidance (线性 +1 MSS / RTT)
loss event	ssthresh = cwnd / 2

10.5 Tahoe vs Reno (slide 67, 70)

	Loss 触发	cwnd 响应	ssthresh	后续
TCP Reno	triple dup ACK	减半 (cwnd/2)	cwnd/2	直接进入 congestion avoidance (线性)
TCP Tahoe	timeout	降到 1 MSS	cwnd/2	从 1 重新 slow start

口诀: Reno 对 dup ACK 减半 (温和); Tahoe 对 timeout 切回 1 (激进)。

PPT slide 70 例: 初始 ssthresh=8, loss 发生在 cwnd=12: – Tahoe: cwnd→1, ssthresh=6, 从 1 慢启动 – Reno: cwnd→6, ssthresh=6, 直接线性增长

11. Bottleneck Link & Delay-based TCP

11.1 Bottleneck Link 问题 (slide 71–72)

经典 TCP 把发送速率推到瓶颈链路饱和: – cwnd 增大 → 瓶颈队列堆积, RTT 增大但 throughput 不再增 – 经典 TCP 看丢包才退让 → 队列长期接近满 (buffer bloat)

Insight: increasing TCP rate at congested bottleneck does not increase throughput, but does increase RTT.
Goal: keep the end-end pipe just full, but not fuller .

11.2 Delay-based 机制 (slide 73–74)

不等丢包, 用 RTT 变化作为拥塞信号:

量	公式
RTT_min	历史观测最小 RTT (无拥塞路径)
uncongested throughput	cwnd / RTT_min
measured throughput	(bytes sent in last RTT) / RTT_measured

条件	判断	行为
measured ≈ uncongested	路径未拥塞	cwnd 线性 ↑
measured << uncongested	路径拥塞 (队列堆积)	cwnd 线性 ↓

优势: congestion control without inducing/forcing loss; 高吞吐 + 低延迟。
代表: TCP Vegas / Google BBR (部署于 Google 内部骨干网)。

11.3 Loss-based vs Delay-based 对比

对比项	Loss-based (Reno/CUBIC)	Delay-based (BBR)
拥塞信号	packet loss/timeout/dup ACK	RTT 增大
是否需丢包才降速	是	否 (proactive)
队列状态	长期接近满	保持低占用

12. TCP Fairness (slide 75)

Fairness goal: K TCP sessions sharing bottleneck of bandwidth R → each get R/K.

AIMD 收敛到公平的直觉: – A 速率 > B → A 先撞拥塞先减半 → A 的份额下降, B 上升 – 双方各自 AIMD 迭代 → 最终趋向 R/2 + R/2

前提: 相同 RTT、MSS、路径。现实偏差: 浏览器多并行 TCP 连接 → 占据更多份额。

13. 关键计算与速查

13.1 给定 Seq 与数据长度, 求 ACK

$$\text{ACK} = \text{Seq} + L$$

例: Host A 发 Seq=500, L=100 → Host B 回 ACK = 600。

13.2 sender window

$$\min(\text{rwnd}, \text{cwnd})$$

13.3 TCP rate

$$\text{rate} \approx \text{cwnd} / \text{RTT}$$

13.4 cwnd 演化决策树

cwnd < ssthresh: slow start, cwnd ×2 / RTT	
cwnd ≥ ssthresh: congestion avoidance, cwnd +1 MSS / RTT	
loss by 3 dup ACK (Reno): ssthresh = cwnd/2; cwnd = cwnd/2; → CA	
loss by timeout (Tahoe): ssthresh = cwnd/2; cwnd = 1 MSS; → SS	

13.5 协议演进一句话总览

rdt1.0	完美信道 → 直接收发
rdt2.0	bit error → checksum + ACK/NAK + retransmit
rdt2.1	ACK/NAK 坏 → seq # (0/1)
rdt2.2	丢 NAK → ACK 带 seq #: duplicate ACK = NAK
rdt3.0	+ 丢包 → countdown timer + timeout
GBN	cumulative ACK + 丢弃失序 + 全部重传
SR	individual ACK + 缓存失序 + 只重传缺的
TCP	字节流 + 累积 ACK + 失序缓存 + fast retransmit + flow + congest

13.6 三层窗口在 TCP 中的角色

窗口	谁维护	限制什么	解决
rwnd	receiver 通告	sender in-flight 字节	flow control
cwnd	sender 自维护	sender in-flight 字节	congestion control
send window	sender	min(rwnd, cwnd)	实际发送上限

COMP5311 Lecture 8 + 9 Cheat Sheet — Network Layer (Data Plane + Routing)

对应课件: Lecture 8 (Router 内部、IP 起步) + Lecture 9 (DHCP、CIDR、NAT、IPv6、OpenFlow、Routing/Dijkstra) 范围: data plane vs control plane, router 内部硬件、forwarding (destination-based / generalized)、OpenFlow、IP addressing、subnets、CIDR、DHCP、route aggregation、NAT、IPv6、tunneling、routing algorithm 分类、Dijkstra (含两个完整算例)

1. Network Layer 总览

1.1 核心使命

Move transport-layer **segments** from sending host to receiving host.

角色	数据单位	行为
Sender host	segment	加 IP header → datagram, 下交链路层
Router	datagram	检查 IP header, 从 input port 搬到 output port
Receiver host	datagram → segment	剥 IP header, 上交传输层

Network layer 协议存在于**每一个** Internet 设备上 (hosts + routers); transport/application 仅在 hosts。Router 协议栈只到 network layer。

1.2 Data Plane vs Control Plane

	Data plane	Control plane
任务	Forwarding: input → output (per-packet)	Routing: 全网计算路径
范围	单个 router 本地	network-wide
时间尺度	纳秒级 (硬件)	毫秒级 (软件)
输出	单包送出哪个端口	生成 forwarding table

口诀: Routing builds the table; Forwarding uses the table.

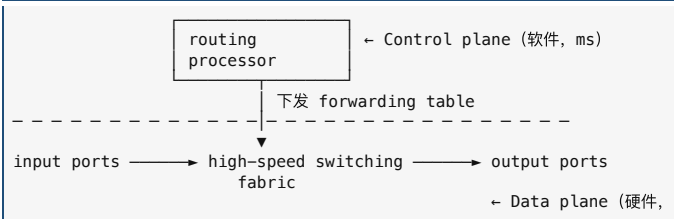
1.3 Control plane 两种实现

方式	说明
Per-router (传统)	每个 router 自跑路由协议 (OSPF/RIP/BGP), 交换信息算路由表
SDN	集中 controller 算好所有 router 的转发表, 下发给各 router; router 只做 data plane

1.4 Forwarding vs Routing 对比

	Forwarding	Routing
中文	转发	路由
Scope	local, per-router	network-wide
回答	这个包从哪个 output 出?	source 到 destination 走哪条 path?
类比	在十字路口选方向	规划整条路线

2. Router 内部架构



四大组成: Input ports / Switching fabric / Output ports / Routing processor。

3. Input Port

3.1 三步流水线

[line termination]	→	[link layer protocol (recv)]	→	[lookup, forward]
物理层 (bit)		链路层 (frame, 如 Ethernet)		网络层 (查表)

3.2 Decentralized switching (关键术语)

- forwarding table 副本预先存在 input port 自己的 memory 中
- 每个 input port 本地完成查表 → 不必每次问中央 routing processor → 并行、快
- 抽象: “match + action”——匹配 header 字段 → 执行动作 (送到某 output port)
- 目标: line speed (处理速度 ≥ 链路速率)

3.3 两种 forwarding 范式

范式	match 看什么	action
Destination-based forwarding (传统)	仅 dest IP	选 output port
Generalized forwarding (SDN)	任意 header 字段组合	drop / forward / modify / log / send to controller

3.4 Input port queueing

当 fabric 跟不上 input rate 时, 包在 input port 排队 → 引发 HOL blocking (见 §5)。

4. Switching Fabric

4.1 衡量: 以 line rate 倍数表示

概念	含义
line rate	单条链路速率 (如 100 Gbps)
switching rate	fabric 内部搬运速率, 常说成“line rate 的 N 倍”
理想	N inputs → 需 N × line rate (避免 input queueing)

4.2 三种 fabric 对比

类型	工作方式	瓶颈	代表
Memory (一代)	CPU 控制, 包进出内存两次	memory bandwidth (≤B/2)	早期 PC 路由器
Bus	input → 共享 bus → output; 不经 CPU	bus contention (一次一个包)	Cisco 5600 (32 Gbps, access router 够用)

类型	工作方式	瓶颈	代表
Interconnection network (crossbar / multistage / Clos)	N×N 开关阵列; 多对不冲突的 input-output 同时传输	接近无瓶颈	高端核心 router

4.3 Crossbar + Cell + 并行传输

- Multistage switch: 用多级小 crossbar 拼成大 N×N, 硬件成本 O(N log N)
- Parallel transmission: datagram 切成定长 cell → 多通路并行 → 出口 reassemble
- 切定长 cell 的好处: 硬件简单、调度整齐、并行度高

5. Input Queueing 与 HOL Blocking

5.1 何时发生

| Fabric 比 input ports 慢 → input 队列堆积 → 排队延迟 + buffer overflow 丢包。

5.2 HOL Blocking 定义

| Queued datagram at front of queue prevents others in queue from moving forward.

| input 队列: [A → out2][B → out1][C → out3]
A 与另一 input 包冲突 out2 → A 等
B 想去 out1 (空闲)、C 想去 out3 (空闲) → 都被 A 堵住

→ 即使其他 output 空闲, 整队卡住。理论上限: input port 利用率被限制到 ~58%。

5.3 解决: Virtual Output Queues (VOQ)

每个 input port 给每个 output 单独建一条队列 → A 被堵不影响 B、C → 彻底消除 HOL。

| HOL blocking 是 input queueing 专属问题; output queueing 没有 HOL (队列里包目的地都一样)。

6. Output Port

6.1 流水线

| fabric → [datagram buffer / queue] → [link layer protocol (send)]

6.2 何时排队 + 后果

| Buffering required when datagrams arrive from fabric faster than link transmission rate. → queueing (delay) and loss due to output port buffer overflow!

这是 Internet 绝大多数丢包和拥塞延迟的物理来源 (不是链路传输错误)。

6.3 Buffer Management (drop / mark)

策略	机制	评价
Tail drop	buffer 满, 丢新到	简单; 可能引起 TCP 全局同步
Priority drop	按优先级丢	保高优先级
ECN (marking)	不丢, header 标记拥塞 → sender 减速	network-assisted CC
AQM / RED	还没满就概率性提前丢	避免 buffer bloat

6.4 Scheduling Discipline

策略	机制	公平性 / 优先级
FCFS / FIFO	按到达顺序发	无
Priority	高优先级队列空才发低; 同级 FCFS; 任何 header 字段可分类	严格优先级; 低优先级会饿死
Round Robin (RR)	各 class 队列轮流取一个完整包; 空队列跳过	包数公平; 不会饿死
Weighted Fair Queueing (WFQ)	RR 的推广: class i 按权重 w_i 取服务	字节级公平 + 加权

WFQ 公式 (高频考点): 每个 class i 的最小带宽保证:

$$\text{class i 拿到的带宽} \geq \frac{w_i}{\sum_j w_j} \times R$$

- 权重由管理员配置 (不是按队列大小)
- 某 class 空闲时其份额按权重比例分给其他 class
- 不会饿死任何 class

7. Forwarding Table 与 Longest Prefix Matching

7.1 Prefix 是什么

Prefix = IPv4 地址的前 N 位 (仍是 IPv4 的一部分, 不是别的东西)。

| IPv4 (32 bit): 11001000 00010111 00011000 10101010
|—— prefix (24 位) ———|—— host (8 位) —|

两种写法等价: | * 写法 | CIDR 写法 ||——|| 11001000 00010111 00010***
***** | 200.23.16.0/21 |

7.2 Longest Prefix Matching

| 多条 prefix 可能同时匹配一个目的 IP → 取最长那条。

NAT is **here to stay**: home/institutional nets、4G/5G cellular nets 大量使用。

13. IPv6

13.1 IPv4 vs IPv6 Header

字段	IPv4	IPv6
Address	32 bit	128 bit (8 组 4 位十六进制)
version	4	6
ToS	diffserv + ECN	priority (流内优先级)
flow label	—	新增: 标识同一“flow”
total length	total length	payload len
TTL	TTL	hop limit
protocol	upper-layer	next hdr (extension 链)
checksum	✓	取消
fragmentation	✓ (router 可分片)	取消 (router 不分片)
options	header 内	取消 (移到 next-header extension)

→ IPv6 header **更精简**，加速 router 处理。

13.2 Tunneling (IPv4-IPv6 共存)

“**Packet within a packet**”: IPv6 datagram 作为 IPv4 datagram 的 payload，穿过 IPv4-only 路由器，到下一个 IPv6 路由器再剥壳。

14. Routing Protocol: 图抽象与算法分类

14.1 目标

在发送方到接收方之间，确定 “good” path: least cost / fastest / least congested。

14.2 图抽象 $G = (N, E)$

- N: router 集合
- E: link 集合
- $c_{a,b}$: 直连链路代价; 非邻居 = ∞
- 代价通常反比于带宽或反比于 (拥塞度的倒数)

14.3 算法分类 (两个独立维度)

维度	类型	代表
信息分布	Global (全拓扑)	Link State = Dijkstra (OSPF)
信息分布	Decentralized (仅邻居)	Distance Vector = Bellman-Ford (RIP)
变化速度	Static	路径变化慢
变化速度	Dynamic	周期更新或链路成本变化时立即更新

15. Dijkstra 算法 (Link-State)

15.1 概念

- Centralized: 所有节点通过 link state broadcast 拿到全网拓扑
- 从 source 算到所有其他节点最短路径
- 结果 = source 的 forwarding table
- Iterative: k 轮后已知到 k 个节点的最短路径
- 复杂度: 朴素 $O(n^2)$; binary heap $O(n \log n)$

15.2 符号

符号	含义
$c_{\{x,y\}}$	直连链路代价; 非邻居 = ∞
$D(v)$	source 到 v 的当前最佳估计
$p(v)$	source 到 v 路径上 v 的前驱节点 (用来回溯)
N'	已确定最短路径的节点集合

15.3 算法 (背诵版)

```
Init:
  N' = {source}
  for each node v:
    if v 邻居 source: D(v) = c_{source,v}, p(v) = source
    else: D(v) = ∞

Loop until N' 包含所有节点:
  1. 在 N' 外找 D(w) 最小的 w
  2. 把 w 加入 N'
  3. 对 w 的每个邻居 v 且 v ∉ N':
      D(v) = min( D(v), D(w) + c_{w,v} )
      (若更新了 D(v), 同时更新 p(v) = w)
```

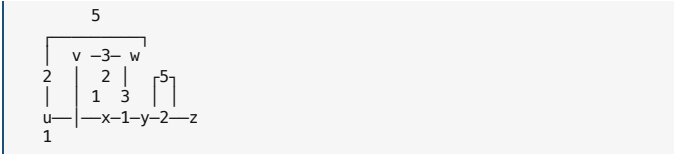
15.4 容易扣分的坑

坑	提醒
忘记同步更新 p(v)	每次更新 D(v) 时立即记 p(v) = w
误更新所有节点	只更新刚加入节点 w 的邻居 (且不在 N')
不取 min	必须 $D(v) = \min(\text{旧}, \text{新})$

坑	提醒
Tie 处理	任选一个，保持一致

16. Dijkstra 例题 1 (标准模板, Slide 62–63)

16.1 拓扑



边权: $u-v=2$, $u-x=1$, $u-w=5$, $v-x=2$, $v-w=3$, $x-w=3$, $x-y=1$, $w-y=1$, $y-z=2$, $w-z=5$

起点: u

16.2 完整迭代表

Step	N'	$D(v),p(v)$	$D(w),p(w)$	$D(x),p(x)$	$D(y),p(y)$	$D(z),p(z)$	加入
0	{u}	2,u	5,u	1,u	∞	∞	x
1	{u,x}	2,u	4,x	—	2,x	∞	y (tie 选 y)
2	{u,x,y}	2,u	3,y	—	—	4,y	v
3	{u,x,y,v}	—	3,y	—	—	4,y	w
4	{u,x,y,v,w}	—	—	—	—	4,y	z
5	{u,x,y,v,w,z}	—	—	—	—	—	done

16.3 推演说明 (关键步骤)

- Step 0: 仅 u 邻居有 D; x 最小 (1) → 加 x
- Step 1: 更新 x 邻居 v/w/y:
 - v: $\min(2, 1+2) = 2$ (不变)
 - w: $\min(5, 1+3) = 4$, $p=w$
 - y: $\min(\infty, 1+1) = 2$, $p=x$
- Step 2 (加 y): 更新 y 邻居 w/z:
 - w: $\min(4, 2+1) = 3$, $p=y$
 - z: $\min(\infty, 2+2) = 4$, $p=y$
- Step 3 (加 v): 更新 v 邻居 w: $\min(3, 2+3) = 3$ (不变)
- Step 4 (加 w): 更新 w 邻居 z: $\min(4, 3+5) = 4$ (不变)

16.4 最终结果

最短代价: $v=2$, $w=3$, $x=1$, $y=2$, $z=4$

回溯路径 (用 p() 倒推): $-v: u \rightarrow v - x: u \rightarrow x - y: u \rightarrow x \rightarrow y - w: u \rightarrow x \rightarrow y \rightarrow w - z: u \rightarrow x \rightarrow y \rightarrow z$

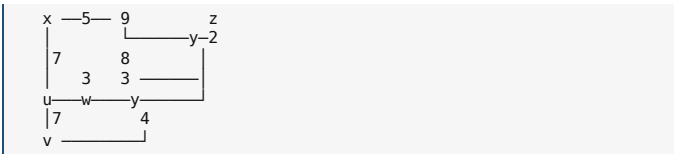
Forwarding table @ u (每个 dest 对应第一跳):

destination	outgoing link
v	(u,v)
x	(u,x)
y	(u,x)
w	(u,x)
z	(u,x)

→ 除直连 v 外，全部经过 x。

17. Dijkstra 例题 2 (Slide 64, 复杂版)

17.1 拓扑



边权 (PPT 给定): $u-v=7$, $u-w=3$, $u-x=5$, $x-w=4$, $x-y=7$, $x-z=9$, $w-y=8$, $w-z=3$, $y-z=2$

起点: u

17.2 完整迭代表

Step	N'	$D(v),p(v)$	$D(w),p(w)$	$D(x),p(x)$	$D(y),p(y)$	$D(z),p(z)$	加入
0	{u}	7,u	3,u	5,u	∞	∞	w
1	{u,w}	6,w	—	5,u	11,w	∞	x
2	{u,w,x}	6,w	—	—	11,w	14,x	v
3	{u,w,x,v}	—	—	—	10,v	14,x	y
4	{u,w,x,v,y}	—	—	—	—	12,y	z
5	{u,w,x,v,y,z}	—	—	—	—	—	done

17.3 推演要点

- Step 0: u 邻居 $v=7$, $w=3$, $x=5$; w 最小 → 加 w
- Step 1 (加 w): 更新 w 邻居 v/x/y:

- o v: $\min(7, 3+3) = 6$, $p=w$
- o x: $\min(5, 3+4) = 5$ (不变)
- o y: $\min(\infty, 3+8) = 11$, $p=w \rightarrow x$ 最小 (5) $\rightarrow$ 加 x
- Step 2 (加 x): 更新 x 邻居 y/z:
 - o y: $\min(11, 5+7) = 11$ (不变)
 - o z: $\min(\infty, 5+9) = 14$, $p=x \rightarrow v$ 最小 (6) $\rightarrow$ 加 v
- Step 3 (加 v): 更新 v 邻居 y:
 - o y: $\min(11, 6+4) = 10$, $p=v \rightarrow y$ 最小 (10) $\rightarrow$ 加 y
- Step 4 (加 y): 更新 y 邻居 z:
 - o z: $\min(14, 10+2) = 12$, $p=y \rightarrow$ 加 z

17.4 最终结果

最短代价: $v=6, w=3, x=5, y=10, z=12$

回溯: - v: $p(v)=w, p(w)=u \rightarrow u \rightarrow w \rightarrow v - w: u \rightarrow w - x: u \rightarrow x - y: p(y)=v, p(v)=w, p(w)=u \rightarrow u \rightarrow w \rightarrow v \rightarrow y - z: p(z)=y \rightarrow u \rightarrow w \rightarrow v \rightarrow y \rightarrow z$

Forwarding table @ u:

destination	outgoing link
w	(u,w)
x	(u,x)
v	(u,w)
y	(u,w)
z	(u,w)

18. 高频考点速查

18.1 CIDR 地址数计算

a.b.c.d/x 含 $2^{(32-x)}$ 个 IP。 - /23 $\rightarrow 2^9 = 512$ - /24 $\rightarrow 2^8 = 256$ - /20 $\rightarrow 2^{12} = 4096$

18.2 CIDR 分配题模板

给定需要 N 个地址 $\rightarrow$ 找最小 k 使 $2^k \geq N \rightarrow$ prefix 长度 = $32 - k$ 。 - 需 2000 $\rightarrow 2^{11} = 2048 \geq 2000 \rightarrow /21$

18.3 NAT 转换答题模板

方向	操作
Outgoing	(LAN_priv_IP : priv_port) $\rightarrow$ (NAT_pub_IP : new_port#), NAT 表记上映射
Incoming	用 NAT 表反查, 把 dest 改回 (priv_IP : priv_port)

要点: ① 替换 source IP/port (出) 或 dest IP/port (入); ② 同一公网 port# 唯一标识一条内部连接。

18.4 IPv4 vs IPv6 关键区别

	IPv4	IPv6
地址长度	32 bit	128 bit
Header checksum	有	无
Router 分片	可	不可
Options	header 内	next-header extension

18.5 OpenFlow / match+action 一句话

不同设备 (router/switch/firewall/NAT) 在 match+action 抽象下都是同一种东西, 区别仅在 flow table 规则不同——这是 SDN 的统一性。

18.6 Dijkstra 答题步骤口诀

1. 画表头: Step | N' | D(v), p(v) | D(w), p(w) | ...
2. Init: N'={source}, 邻居填直连 cost, 非邻居填 ∞
3. 循环: 找 N' 外 D 最小 $\rightarrow$ 加入 $\rightarrow$ 更新该节点邻居的 D / p
4. 重复直到 N' 含所有节点
5. 用 p() 回溯路径
6. 写 forwarding table (每个 dest 对应第一跳)

19. 一图串起整个 Network Layer

Control Plane (软件, 毫秒级) - 跑 Routing 协议: Dijkstra (LS) / Bellman-Ford (DV) - Routing processor 算出 forwarding table, 下发到 data plane

Data Plane (硬件, 纳秒级 —— 单 router 内部)

模块	职责
Input ports	line termination · link layer · lookup (LPM/TCAM) · match+action · input queueing (HOL blocking $\rightarrow$ VOQ)
Switching Fabric	memory / bus / interconnection 三种实现 · cell + 并行传输
Output ports	buffer (drop policy) · scheduling (FCFS/Priority/RR/WFQ) · link-layer · line term · 排队+丢包

支撑大规模 Internet 的协议层面

- IP addressing (interface, subnet)
- CIDR (a.b.c.d/x, longest prefix matching)

- DHCP (动态分配, DORA + broadcast)
- Hierarchical addressing & route aggregation
- NAT (private IP + port translation)
- IPv6 (128-bit, simplified header, tunneling 过渡)

抽象演进: destination-based forwarding $\rightarrow$ generalized forwarding (match+action) $\rightarrow$ OpenFlow $\rightarrow$ SDN

20. 跨三章解答题速答 (考前最后一遍)

每条用 1-3 句话给出答题骨架; 细节回到对应章节正文。

20.1 Application Layer

Q1 HTTP persistent vs non-persistent 响应时间 - Non-persistent: 每个 object 2 RTT + 传输 (TCP setup 1 RTT + 请求/首字节 1 RTT); Persistent option 1 (逐个) 每 object 1 RTT; Persistent + pipelining (option 2) N 个 object 总共 $\approx 1 RTT + N \times$ 传输。

Q2 Cookies 怎么用来追踪 - 首次访问 $\rightarrow$ server 设 Set-Cookie: $\rightarrow$ 浏览器存 $\rightarrow$ 下次访问该域/第三方域时自动带上 $\rightarrow$ 构建用户画像; 隐私问题主要来自 third-party cookie。

Q3 Web Cache 三个公式 - 命中率 h; access link 新流量 = $(1-h) \times$ 原始; 端侧平均延迟 = $(1-h) \cdot D_{origin} + h \cdot D_{cache}$; cache 既减延迟又减带宽。

Q4 DNS iterative vs recursive - Iterative: 被问者返回下一站地址, 发起者继续问。Recursive: 被问者替你继续递归直到答案。实际 Internet: client $\rightarrow$ Local DNS 递归; Local $\rightarrow$ Root/TLD/Authoritative 迭代。

Q5 DNS 为何用 UDP - 查询小、对延迟敏感; UDP 无握手 $\rightarrow$ 来回; 丢失靠应用层重试; 少量大查询 (zone transfer) 才用 TCP。

Q6 P2P 文件分发时间下界 - $D_{P2P} \geq \max\{F/u_s, F/d_{min}, NF/(u_s + \sum u_i)\}$; 自扩展性: peer 越多总上传带宽越大, 时间不随 N 线性增长。

Q7 Client-server vs P2P - C-S: $D_{CS} \geq \max\{NF/u_s, F/d_{min}\}$ 与 N 成正比; P2P 见 Q6。

20.2 Transport Layer (RDT & TCP)

Q8 rdt 各版本演进 - 1.0 $\rightarrow$ 2.0: bit error $\rightarrow$ checksum + ACK/NAK; 2.0 $\rightarrow$ 2.1: ACK/NAK 也可能坏 $\rightarrow$ 加 0/1 序号; 2.1 $\rightarrow$ 2.2: 用 duplicate ACK 取代 NAK; 2.2 $\rightarrow$ 3.0: 增加丢包 $\rightarrow$ countdown timer + 超时重传。

Q9 rdt2.0 fatal flaw - ACK/NAK 坏时 sender 不知 receiver 状态: 重传可能 duplicate, 不重传可能丢失, 没有序号无法区分。rdt2.1 用 0/1 序号修复。

Q10 GBN vs SR - GBN: 累积 ACK + 失序丢弃 + 超时重传整个窗口; SR: 逐个 ACK + 失序缓存 + 只重传缺的; SR 窗口 $\leq$ 序号空间/2。

Q11 3-way 而非 2-way 的理由 - 防 half-open (旧 SYN 延迟到达建出无 client 的连接) + 防 duplicate data (旧 data 落入新连接); 3-way 同步 ISN, 确认双方都能 send/receive。

Q12 三次握手每步双方知道什么 - 1st 后: server 知道 client $\rightarrow$ server 通; 2nd 后: client 知双向通, server 仍不知 client 是否能收; 3rd 后: 双方都知双向通。

Q13 Fast Retransmit 触发 / 为什么是 3 - 收到 3 个重复 ACK 立即重传 smallest seq #; 3 个 dup 说明已有 3 个后续 segment 到达, 缺的高概率丢失; 1-2 个可能只是 reorder。

Q14 Flow vs Congestion control - Flow: 保护 receiver buffer, 用 rwnd, receiver 通告; Congestion: 保护 network, 用 cwnd, sender 从 loss/delay 推断。Sender 实际窗口 = $\min(rwnd, cwnd)$ 。

Q15 AIMD 为什么不对称 (+1 / +2) - 进取要慢慢避免冲击网络; 退让要快速缓解拥堵; 两个 AIMD 流会自动收敛到公平共享 $R/2 + R/2$ 。

Q16 Slow start vs Congestion avoidance - $cwnd \leq ssthresh$ 指数 (每 RTT $\times 2$); $cwnd \geq ssthresh$ 线性 (+1 MSS / RTT); loss 时 $ssthresh = cwnd/2$ 。

Q17 Tahoe vs Reno - Reno: 3 dup ACK $\rightarrow$ $cwnd /= 2$, 进入 CA (温和); Tahoe: timeout $\rightarrow$ $cwnd = 1 MSS$, 重新 slow start (激进)。

Q18 TimeoutInterval 公式 - $EstRTT = (1-\alpha) \cdot old + \alpha \cdot Sample$ ($\alpha=0.125$); $DevRTT = (1-\beta) \cdot old + \beta \cdot |Sample - EstRTT|$ ($\beta=0.25$); $Timeout = EstRTT + 4 \cdot DevRTT$ 。

Q19 BBR 是什么 - Delay-based CC: 用 RTT 增大作为拥塞信号, 目标 keep pipe just full but not fuller; 高吞吐 + 低延迟。

20.3 Network Layer

Q20 Forwarding vs Routing - Forwarding: 单 router 本地动作 (per-packet, ns, 硬件); Routing: 全网计算路径 (ms, 软件) $\rightarrow$ 生成 forwarding table。Routing builds the table; forwarding uses it.

Q21 Data vs Control plane - data 执行 forwarding; control 算 routing 并生成 table。SDN 把 control plane 集中到 controller。

Q22 Decentralized switching / match+action / line speed - 每个 input port 在自己 memory 查 forwarding table, match header 字段 $\rightarrow$ action (drop/forward/modify), 目标速度 $\geq$ 链路速率。

Q23 Longest Prefix Matching - 多条 prefix 同时匹配时取最长; 长 prefix 更具体; 硬件用 TCAM 一拍出结果。

Q24 三种 Switching Fabric - Memory (瓶颈 memory bw)、Bus (瓶颈 bus contention, Cisco 5600=32 Gbps)、Interconnection (crossbar/multistage/cell+并行, 最快)。

Q25 HOL blocking - input queueing 时队首被堵 $\rightarrow$ 后面能走的也被堵; 理论利用率上限 ~58%; 解决: VOQ (每个 output 一条独立队列)。

Q26 Output port queueing 后果 - 多 input 打到同一 output 聚合速率 $>$ 链路 $\rightarrow$ 排队 + buffer overflow 丢包; Internet 主要丢包源。

Q27 WFQ 公式 - class i 最小带宽 = $w_i / \sum w_j \times R$; 权重由管理员配置, 不是按队列大小; 空闲份额按比例分给其他 class, 不饿死。

Q28 DHCP DORA + 为什么 broadcast — Discover→Offer→Request→ACK (前两步可跳过); client 没 IP 也不知 server, 源 0.0.0.0 / 目的 255.255.255.255 / MAC FF:FF:FF:FF:FF:FF; DHCP→UDP→IP→Ethernet, 端口 67/68。

Q29 CIDR 地址数 / 分配 — /x 含 $2^{(32-x)}$ 个 IP; 分配 N 个→找最小 k 使 $2^k \geq N \rightarrow \text{prefix}=32-k$ 。例: 2000 → /21 (2048)。

Q30 NAT 工作原理 + 优劣 — LAN 内 private IP, NAT 表 (WAN pub IP : new port) ↔ (LAN priv IP : priv port); 优: 省公网 IP、内部自由、安全; 缺: 违反 e2e、P2P/VoIP 不友好、延迟、troubleshooting 复杂。

Q31 IPv4 vs IPv6 区别 — 32→128 bit; 取消 header checksum / router 分片 / options (移到 next-header); 新增 flow label、hop limit; header 更精简。

Q32 Tunneling — “packet within a packet”: IPv6 datagram 作 IPv4 datagram 的 payload, 穿过 IPv4-only 路由器, 到下一个 IPv6 router 剥壳。

Q33 OpenFlow match+action 统一性 — Router (longest prefix→forward)、Switch (dst MAC→forward/flood)、Firewall (IP/port→permit/deny)、NAT (IP+port→rewrite) 在同一抽象下都是同一种设备, 区别只在 flow table 规则——SDN 基石。

Q34 Dijkstra 手算 — ① $N'=\{\text{source}\}$, 邻居填 cost、其他 ∞ ; ② 找 N' 外 D 最小的 w 加入 N' ; ③ 更新 w 邻居 v: $D(v)=\min(D(v), D(w)+c_{\{w,v\}})$ 同时记 $p(v)=w$; ④ 重复直到全在 N' ; ⑤ 用 p() 回溯。复杂度 $O(n^2)$ 或 $O(n \log n)$ 。

Q35 Web 请求完整流程 — ①DHCP 拿 IP/网关 IP/DNS IP; ②ARP 拿网关 MAC; ③DNS 查目的 IP (包送给网关 MAC); ④TCP 三次握手 (1 RTT); ⑤HTTP GET → response。MAC 是下一跳, IP 是最终目的——网关 MAC 全程不变直到出本网段。

COMP5311 Lecture 6 Cheat Sheet — Tutorial: Socket Programming / Wireshark / VPN

对应课件: Lecture 6 (Tutorial), slides 4–67。本讲实践向, 考点: ① TCP/UDP socket 流程对比; ② Wireshark 识别 TCP 握手; ③ VPN 类型与 IPsec/IKE 两阶段。

1. Socket Programming

1.1 Socket 概念

Socket = 应用进程与传输层之间的接口 (“门”)。应用通过 socket 把数据交给 OS 的协议栈, 由 TCP/UDP 发出。

两类 socket (按传输服务):

Socket type	传输协议	服务模型
SOCK_DGRAM	UDP	unreliable datagram
SOCK_STREAM	TCP	reliable byte stream

1.2 UDP vs TCP socket API 对比 (高频考点)

项目	UDP (无连接)	TCP (面向连接)
连接	无	必须 connect()
Client 发送	sendto(data, dst_addr)	send() / sendall()
Client 接收	recvfrom() → 返回 (data, src_addr)	recv()
Server 监听	bind → recvfrom 循环	bind → listen → accept 循环
每次发送是否带 dst	是	否 (connect 时已定)
可靠/有序	不保证	保证
典型应用	DNS, streaming, games	HTTP, FTP, email

1.3 TCP server 的两个 socket (必考)

Socket	作用
welcoming socket	listen 在固定端口, 接受新连接请求
connection socket	accept() 返回的, 与某一 client 一对一通信

→ 多客户端处理: 每 accept() 一个连接 → 起一个 thread 处理它, 主线程继续 accept。

1.4 答题要点

- “Why UDP sendto instead of send?” → UDP 无连接, 每个 datagram 必须自带目标地址。
- “Welcoming vs connection socket?” → welcoming 等连接; connection 服务已建立的具体 client。

2. Wireshark

2.1 用途与字段

图形化 packet analyzer: 实时抓包或读 PCAP 文件, 把抽象协议变成可见的 packet。

字段	含义
Time / Source / Destination	时间 / 源 IP / 目的 IP
Protocol	TCP / UDP / HTTP / DNS ...
Length	packet 字节数
Info	摘要, 如 [SYN]、GET /

每个 packet 展示分层封装 (自外而内): Frame → Ethernet II → IP → TCP/UDP → HTTP/DNS/...

2.2 常用 filter 思路

Filter	含义
host X	与 X 有关
src host X / dst host X	方向限定
port N / tcp port N	按端口
tcp / udp / http / dns	按协议
and / or / not	组合

2.3 在 Wireshark 中识别 TCP 握手与挥手 (高频)

阶段	Packet 序列
建立 (3-way)	[SYN] → [SYN, ACK] → [ACK]
关闭 (4-way)	[FIN, ACK] → [ACK] → [FIN, ACK] → [ACK]

关闭为何 4 步: TCP 是 full-duplex, 两个方向独立关闭——一边说“我不发了”, 对端先 ACK; 之后对端也说“我也不发了”, 再被 ACK。

3. VPN (Virtual Private Network)

3.1 定义与目标

VPN 在公共 Internet 之上建立加密隧道, 让通信看起来像在私有网络中——不需要租专线。

提供的安全功能 (必背 5 项):

功能	含义
Confidentiality	数据加密, 防窃听
Integrity	检测篡改
Authentication	验证发送者身份
Replay protection	防止旧包重放攻击
Access control	控制谁能访问内部资源

3.2 两种 VPN 类型对比

	Site-to-site VPN	Remote access VPN
连接对象	network ↔ network	user ↔ network
端点	两端 gateway/router	用户设备 + 公司 VPN server
隧道数量	整个 office 共享一条	每用户一条
配置位置	gateway	用户安装 VPN client
典型场景	分支 ↔ 总部	出差/居家员工 ↔ 公司

记忆: Site-to-site = 网到网; Remote access = 人到网。

3.3 VPN 四大关键技术

技术	解决的问题
Tunneling	怎么穿过 public Internet (封装)
Encryption / decryption	让别人看不懂内容
Authentication	确认通信对方身份
Key management	加解密钥如何安全协商

3.4 Tunneling 三种协议角色 (PPT 原话)

角色	含义	类比
Passenger protocol	原始被封装的协议/数据	乘客
Encapsulating protocol	隧道协议 (如 PPTP, IPsec, GRE)	车厢
Carrier protocol	承载隧道穿网的协议 (通常 IP)	道路

3.5 PPTP vs IPsec

	PPTP	IPsec
时代	早期	现代主流
安全性	弱加密、已知漏洞	强
现状	largely superseded	广泛部署
用途	早期 remote access	site-to-site / gateway / host

3.6 IPsec / IKE 两个阶段 (高频考点)

	Phase 1	Phase 2
目标	建立安全控制通道、双方认证	协商真正用于数据传输的加密参数
速度	慢且重 (asymmetric encryption)	快且敏捷 (symmetric)
认证手段	pre-shared key 或 certificate	复用 Phase 1 的密钥

	Phase 1	Phase 2
输出	IKE SA	IPsec SA
直觉	“先安全地商量怎么通信”	“正式保护大量数据传输”

4. 解答题速答

Q1 UDP 为何用 `sendto` 不用 `send` — UDP 无连接，每个 datagram 必须显式带目的 IP+port。

Q2 TCP server welcoming vs connection socket — welcoming 在固定端口 listen 等新连接；accept 后产生 connection socket，与具体某 client 一对一通

信。

Q3 Wireshark 识别 TCP 三次握手 — SYN → SYN/ACK → ACK；关闭是 4-way FIN/ACK → ACK → FIN/ACK → ACK，因为 TCP 双向独立关闭。

Q4 Site-to-site vs remote access — 前者连两个网络（gateway 维护），后者连个人用户与公司网（用户设备维护）。

Q5 Tunneling 是什么 — 一种 encapsulation：把原始 packet 封装进新的外层 packet，沿公共 Internet 传输到隧道另一端再剥壳还原。

Q6 PPTP 为什么过时 — 加密弱、有已知漏洞，已被 IPsec 等现代协议替代。

Q7 IKE Phase 1 vs Phase 2 — Phase 1 慢、做认证、建立 IKE SA（控制通道）；Phase 2 快、用 Phase 1 的密钥协商 IPsec SA（数据通道）。